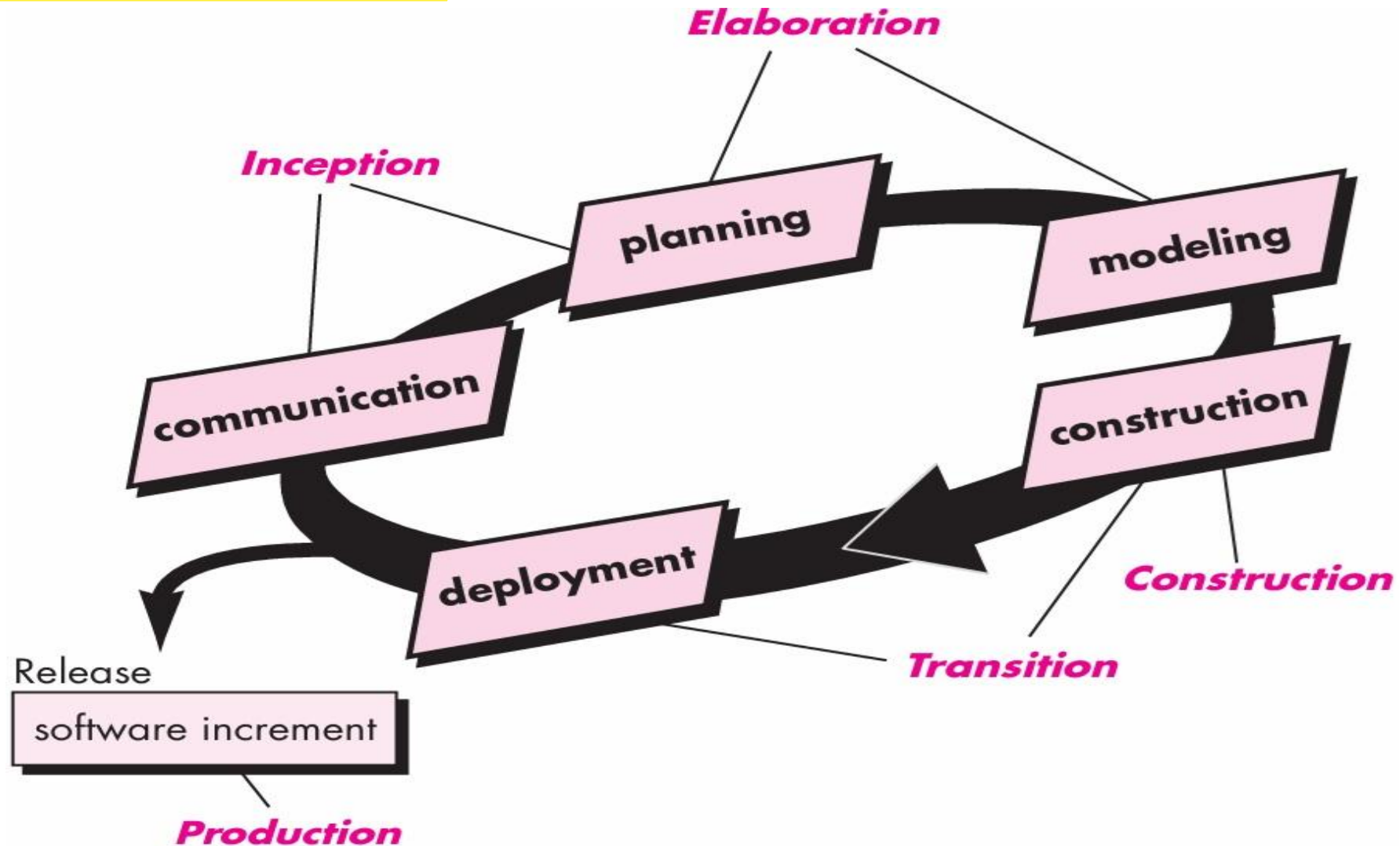


Software Engineering

Dr. Md. Nasim Adnan

Why we model?



Modeling is a proven and well-accepted engineering technique. Unsuccessful software projects fail in their own unique ways, but all successful projects are alike in many ways. There are many elements that contribute to a successful software organization; one common thread is the use of modeling. We do model so that we can better understand the system we are developing.

What is a Model?

- A model is a simplification of reality
- A model provides the blueprints of a system

Benefits of Model

1. Models help us to visualize a system as it is or as we want it to be
2. Models permit us to specify the structure or behaviour of a system
3. Models give us a template that guides us in constructing a system
4. Models document the decisions we have made

Limitation of Human Ability

- The bigger and more complicated a system is, the more important modeling becomes.
- This is because we cannot fully understand the whole system at once.
- Humans have limits in understanding very complex things.
- Modeling helps us focus on a smaller part (subsystem) of the big system, so we can study one specific part at a time.

Principles of Modeling – 1st

- The decision about which models to make greatly affects how we approach a problem and how we design the solution.
- In other words, it's important to choose models carefully and thoughtfully. The right models help us clearly see and understand difficult development problems.

Principles of Modeling – 2nd

Every model can be created with different amounts of detail and accuracy. Sometimes, a fast and simple working model of the user interface is just what is needed; other times, it is necessary to work with the very detailed technical parts. In any case, the best models are those that allow deciding how much detail to show, based on who is looking at the model and why they need to see it.

Principles of Modeling – 3rd

- The best models are connected to reality.
- A Model connected to reality makes it more understandable and presentable. The more abstract a model is the more it needs to be closer to reality.

Principles of Modeling – 4th

No single model is enough by itself. A system that is not simple, is best understood by using a few models that are mostly independent from each other. To fully understand the system, it is important to look at different models. In this way, the models support and complete each other.

Requirement Modeling

- At a technical level, software engineering begins with a series of modeling tasks that lead to a specification of requirements and a design representation for the software to be built.
- The requirements model— actually a set of models—is the first technical representation of a system.

**Scenario-based
models**

e.g.,
use cases
user stories

**Class
models**

e.g.,
class diagrams
collaboration diagrams

**Software
Requirements**

**Behavioral
models**

e.g.,
state diagrams
sequence diagrams

**Flow
models**

e.g.,
DFDs
data models

Scenario-based Modeling

- Although a computer system or product's success is judged in many ways, making users happy is the most important.
- If you understand how the users (and other people involved) want to use the system, your software team can better describe the requirements and create useful models for both requirements and design.
- That's why when using **UML for requirements modeling**, you start by creating scenarios shown as use case diagrams.

UML

- Unified Modeling Language (UML) is a standardized general-purpose modeling language in the field of object-oriented software engineering.
- UML includes a set of graphic notation techniques to create visual models of object-oriented software systems.

UML is a Language for Visualizing

- UML works independently of any programming language.
- One person can write a model using UML, and another person, or even another tool, can interpret that model clearly.
- It's possible to convert a UML model into programming languages like C++, Java, or C#, or even into different types of databases (RDBMS).

Goals of UML

- Visibility
- Extensibility
- Independence
- Object Oriented

Use Case Modeling

- After deciding the project scope (which includes recognizing the need and doing a feasibility study), the process of gathering requirements can start.
- Requirements are collected using different communication methods such as:
 - Interviewing the people involved
 - Having stakeholders fill out questionnaires
 - Watching how the current manual system (if there is one) works before automating it
- Use cases become important during the requirements gathering phase.
- The main goal of writing use cases is to clearly capture the functional requirements of the system being designed.
- Because of this, use cases act as a guide for white box testing to check if the requirements have been met or not

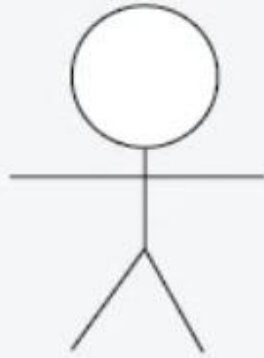
use case in your diagram for
"Room Allocation"



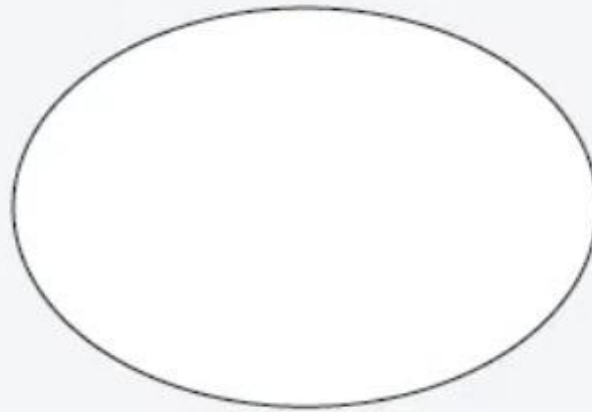
Actors and Use Cases

- The first step in writing a use case is to define the set of “actors” that will be involved in the story. Actors are the different people (or devices) that use the system or product.
- **Actors represent the roles that people (or devices) play as the system operates.** More formally, an actor is anything outside the system that interacts or communicates with it. Each actor has one or more goals they want to achieve by using the system.
- Once actors have been identified, use cases can be developed.

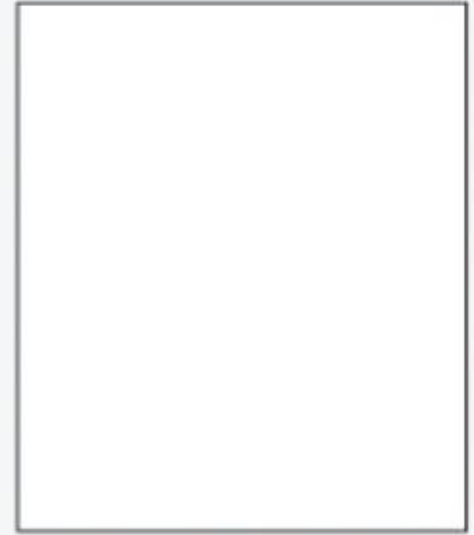
Use Case Diagram Notations



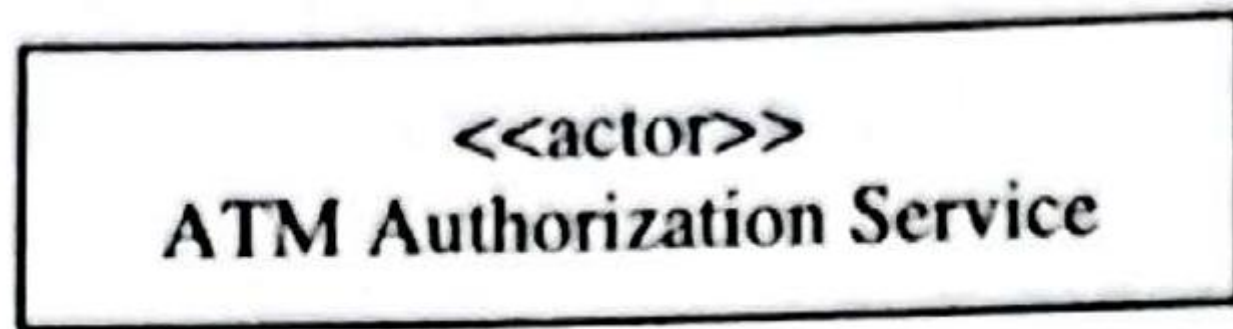
Actor



UseCase

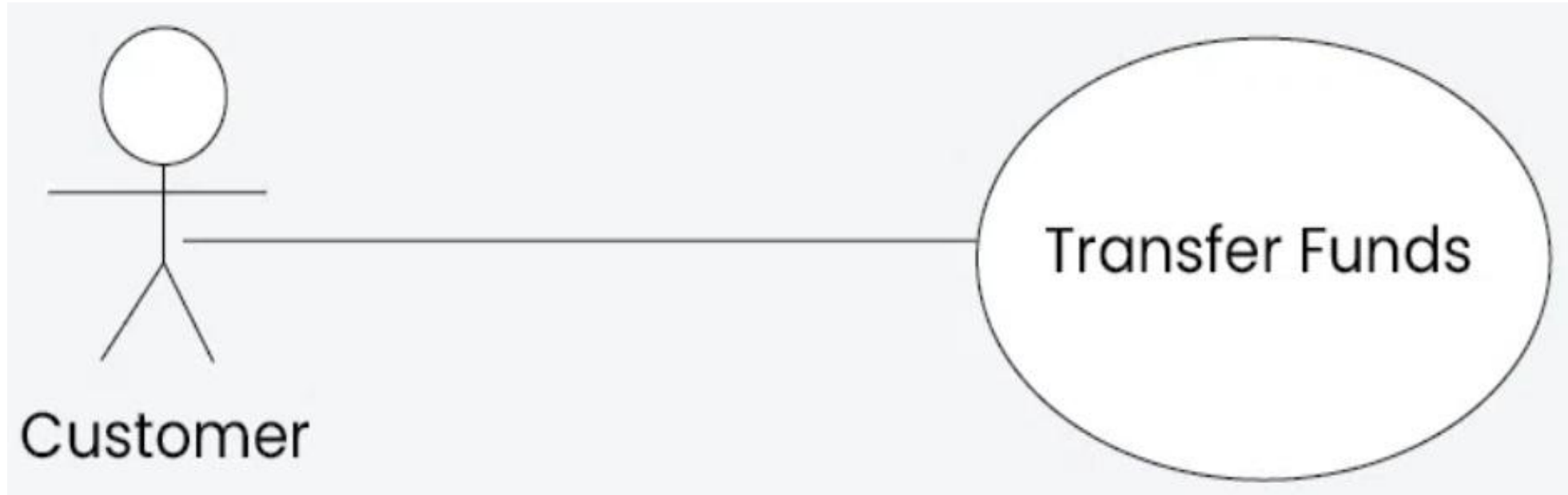


System Boundary



Use Case Relationships

- Association Relationship



- **Include Relationship**

The 'include' relationship is used to help reuse parts of use cases. A designer should not use 'include' if the included use case shows behavior that only applies to one specific use case and cannot be used by others.

For example, in a video library system, customers can either rent or buy a DVD. In both cases, the customer pays with a credit card. So, the use cases 'Process Rental Request' and 'Process Sale Request' can both include the use case 'Handle Credit Card Payment' instead of repeating the same steps twice. This way, the system avoids doing the same thing more than once. Figure 2.3 shows how the 'include' relationship works.

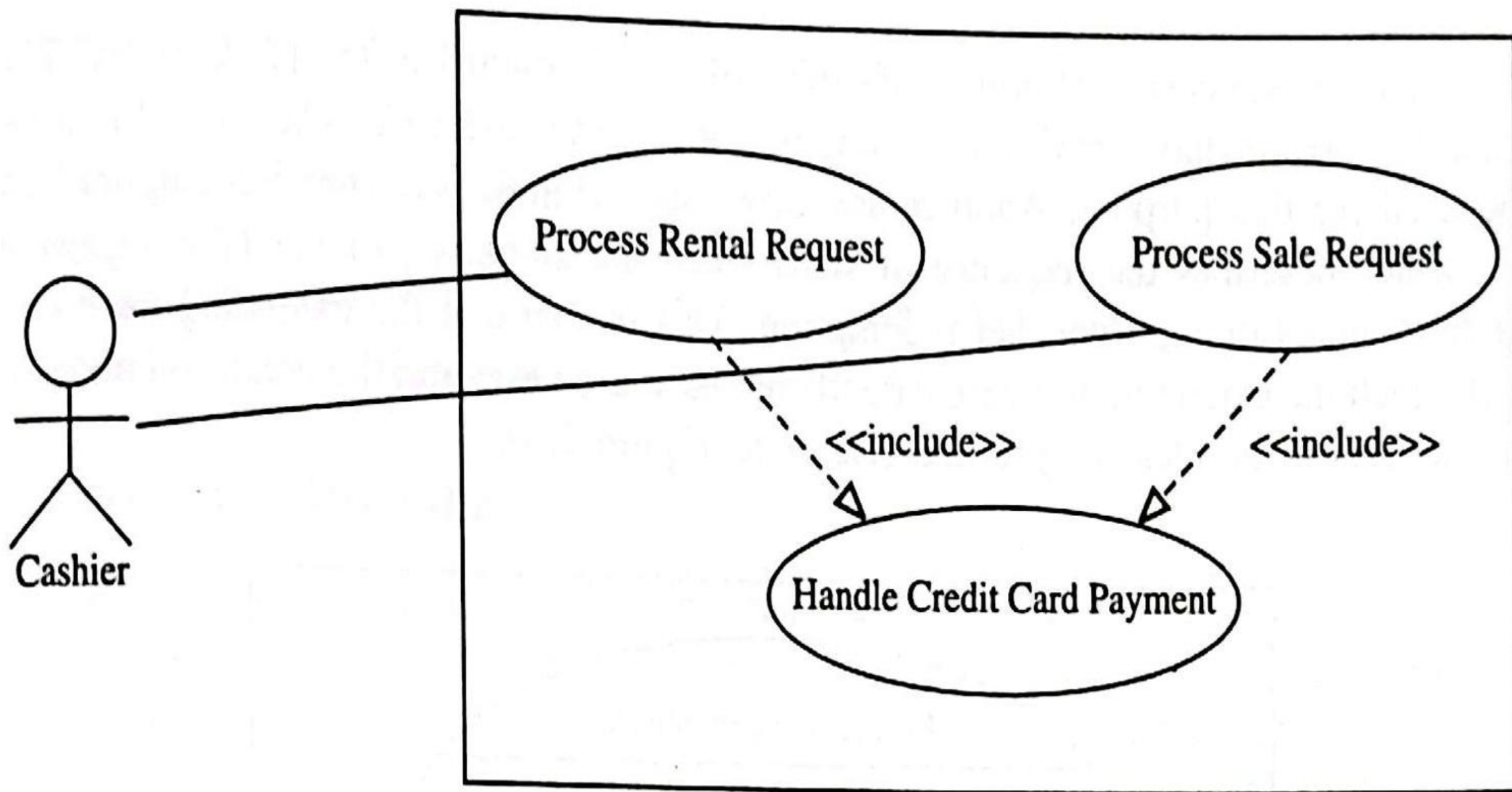
When to use the 'include' relationship:

The 'include' relationship is used to help reuse parts of use cases. A designer should not use 'include' if the included use case shows behavior that only applies to one specific use case and cannot be used by others.

For example, in a video library system, customers can either rent or buy a DVD. In both cases, the customer pays with a credit card. So, the use cases 'Process Rental Request' and 'Process Sale Request' can both include the use case 'Handle Credit Card Payment' instead of repeating the same steps twice. This way, the system avoids doing the same thing more than once. Figure 2.3 shows how the 'include' relationship works.

When to use the 'extend' relationship:

The 'extend' relationship is used when a use case adds extra steps or behavior to another use case, but only in certain situations. It is like an optional addition that happens sometimes, not always. For example, if a customer wants to get a discount during payment, the discount process can be an extension of the normal payment process. So, the main use case is 'Handle Payment,' and the 'Apply Discount' use case extends it when needed.



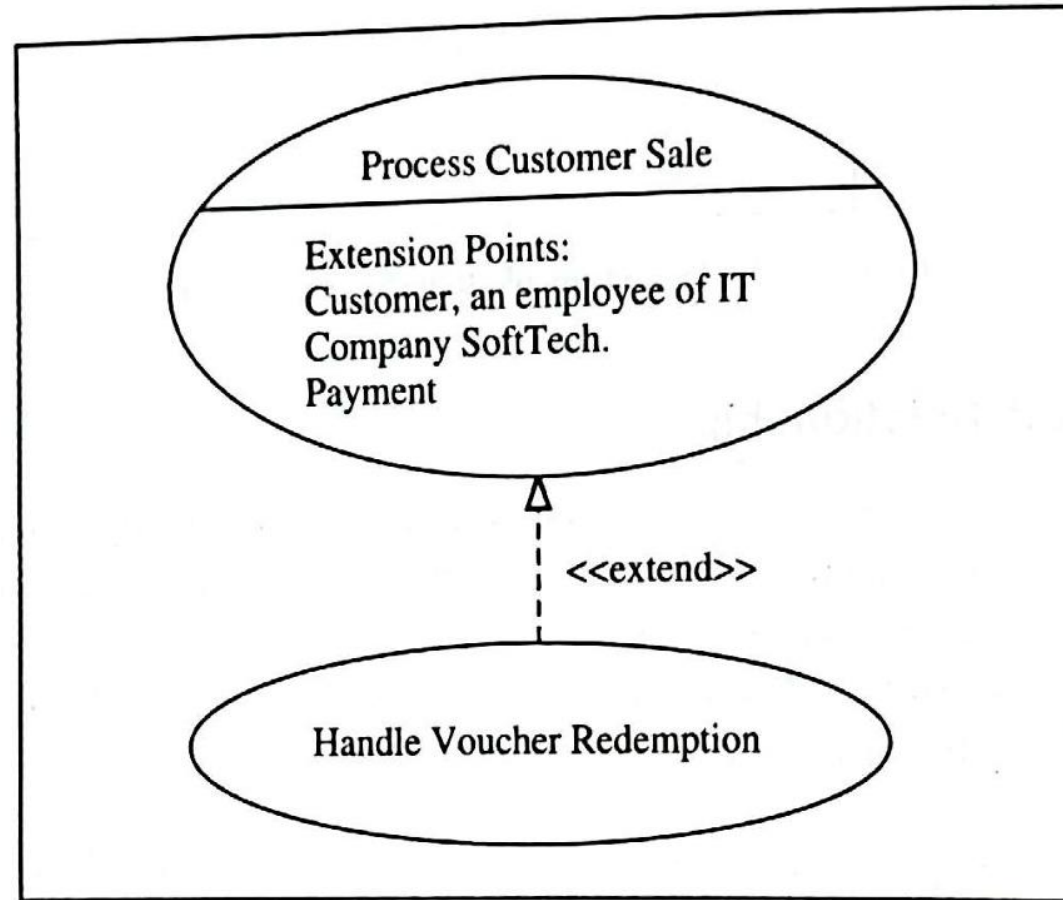
- Extend Relationship

The extend relationship is represented by a dashed arrow with the `<<extend>>` stereotype. The base of the arrow points to the extending use case, while the arrowhead points to the extended use case.

For example, consider an IT company that pays software developers high salaries, partially in gift vouchers redeemable at specific supermarkets. Initially, the supermarket management system was designed to accept only cash payments. However, after signing an agreement with the IT company, the supermarket requested the system to include a feature for accepting payment through gift voucher redemption. Let us assume the use case 'Process Customer Sale' was structured as follows:

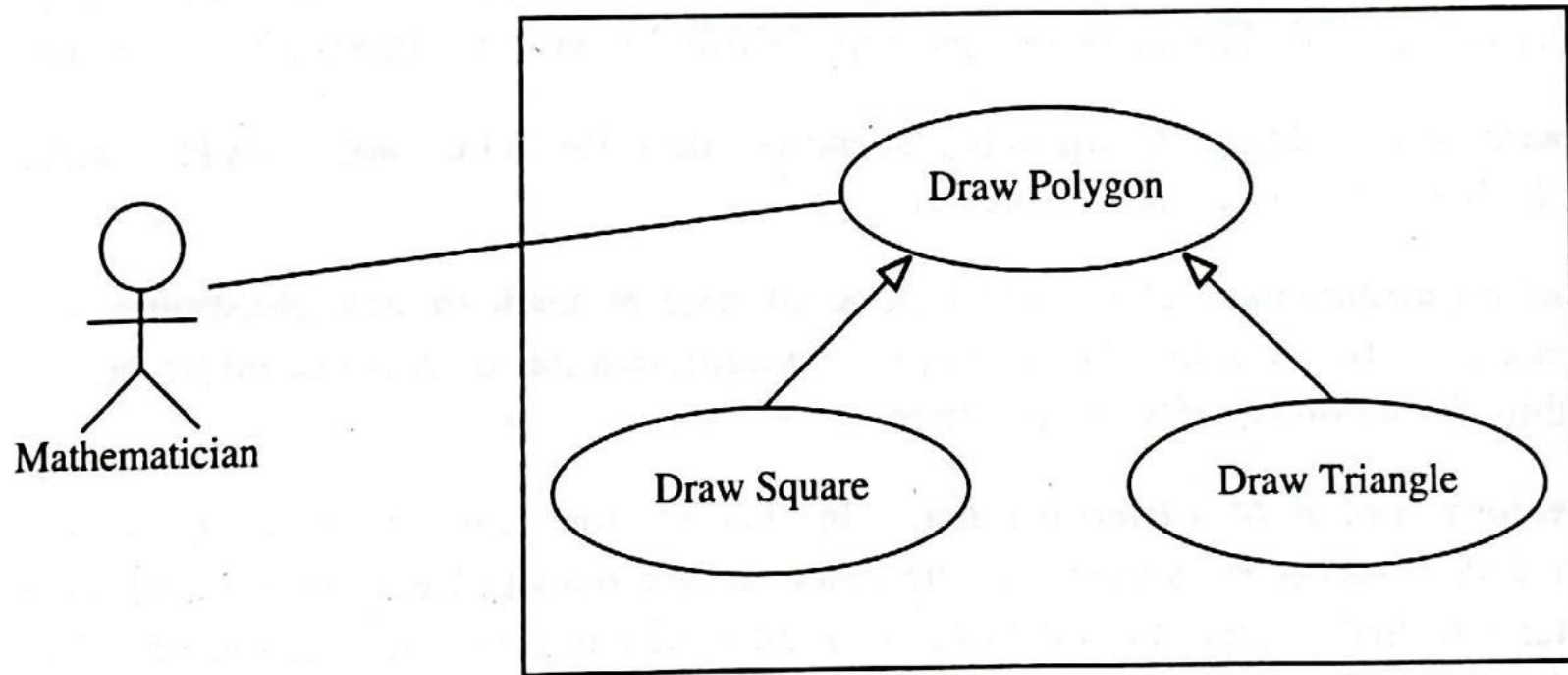
1. Customer arrives at the payment counter with the goods that he intends to buy.
2. Cashier starts a new transaction on the system.
Cashier repeats steps 3 and 4 till all the goods submitted by the customer are over.
3. Cashier submits the bar code of the item to the system with the aid of a bar code scanner.
4. System reads the item's description and its price
5. System displays total amount.
6. Cashier communicates total to the customer and asks for payment.
7. Customer pays the required amount through cash.
8. Cashier commands system to generate receipt.
9. System logs the transaction and generates the receipt.
10. Customer leaves with his goods and the receipt.

Suppose that this use case is an artefact and hence cannot be modified. Yet the store wants to start the voucher redemption scheme with the customers of the IT company. The aim is to add the extra behaviour without changing the original use case. The extend relationship can be used for this purpose. Another use case, say, 'Handle Voucher Redemption' can be written which describes the sequence of steps when the employee of the IT company wants to purchase goods through voucher redemption. This use case is the extending use case. The point at which the extending use case extends the base use case, and the condition under which it does so, has to be clearly specified (Refer to Figure 2.4).

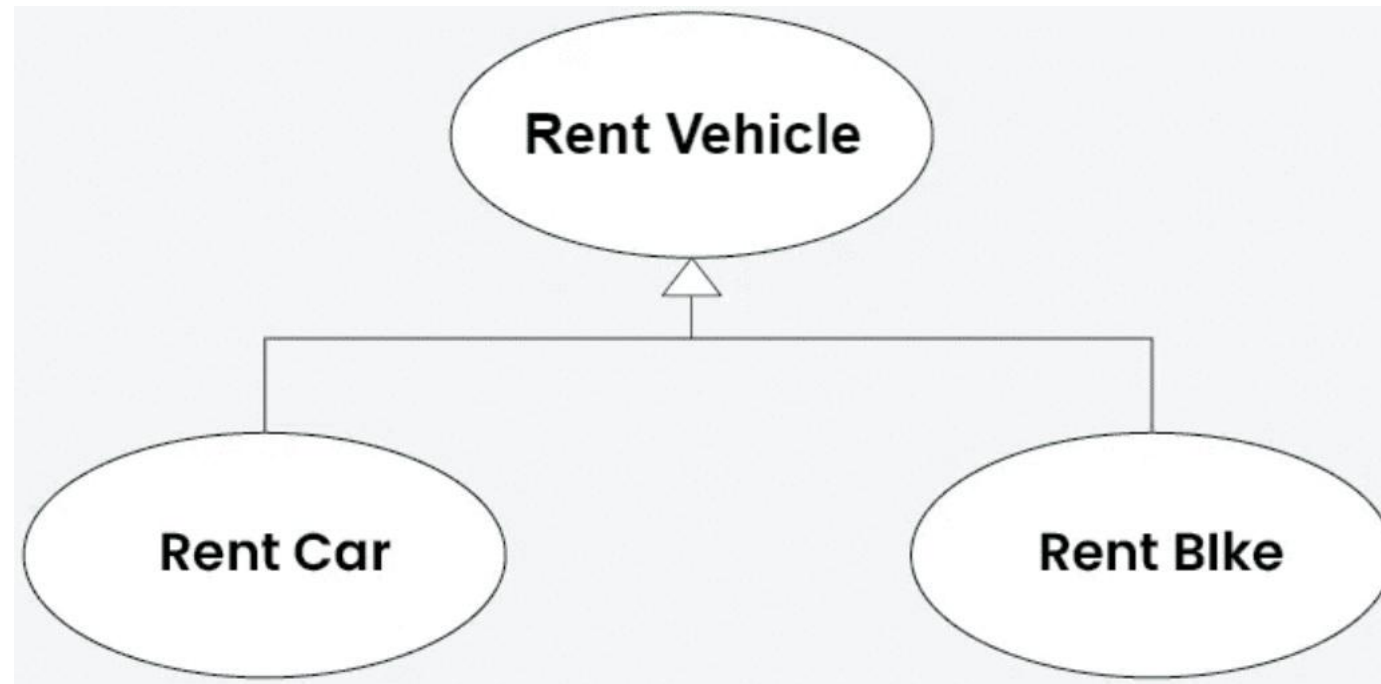


- **Generalization relationship**

Generalization is like an inheritance (parent-child) relationship where a general use case/class has common behavior, and specific ones inherit it while adding their own details.



- Generalization relationship

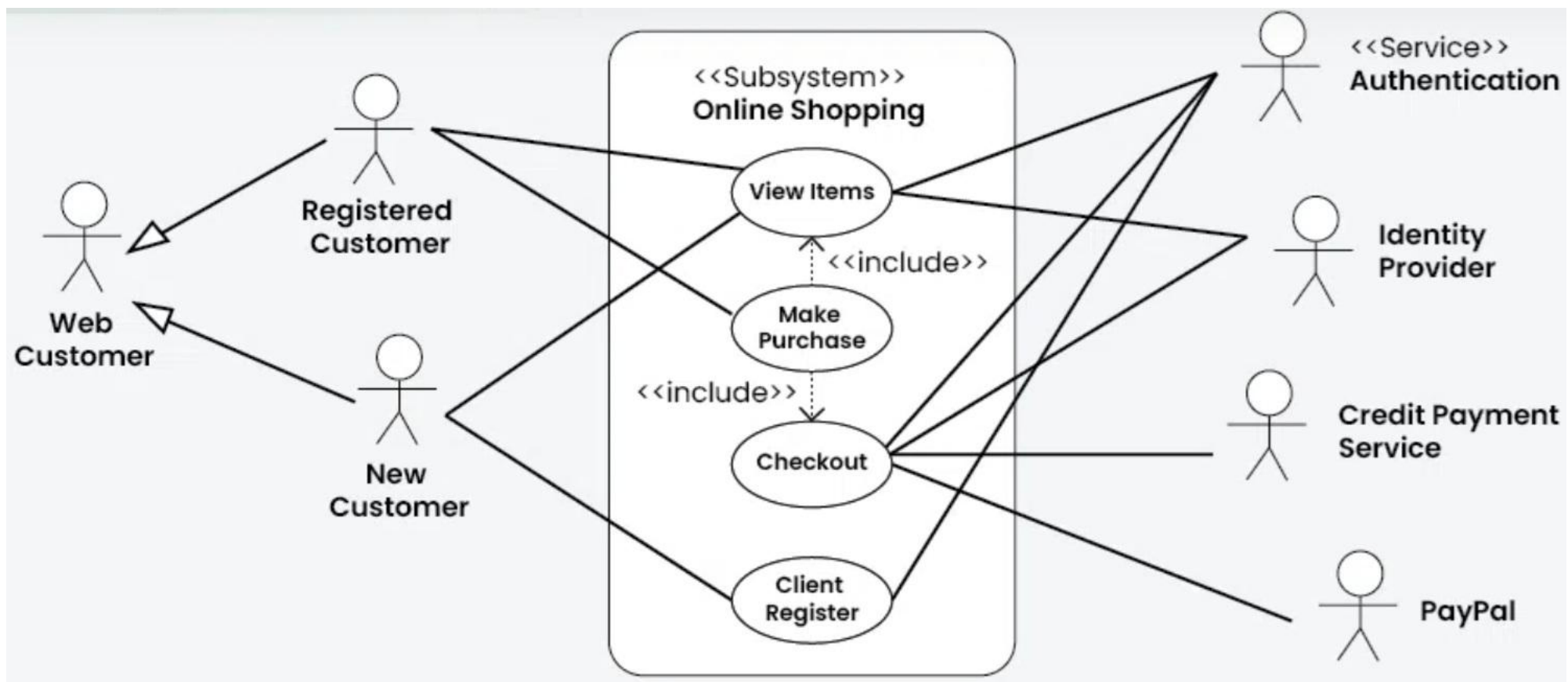


How to draw a Use Case diagram in UML?

- Step 1: Identify Actors
- Step 2: Identify the main functionalities or actions the actors must perform using the system. These are your use cases.
- Step 3: Specify requirements, pre and post conditions
- Step 4: Connect Actors and Use Cases
- Step 5: Add System Boundary
- Step 6: Define Relationships
- Step 7: Review and Refine
- Step 8: Validate

Use Case Diagram for Online Shopping System

- Actors:
 - Customer
 - Admin
- Use Cases:
 - Browse Products
 - Add to Cart
 - Checkout
 - Manage Inventory (Admin)
- Requirements:
 - The Customer can browse products, add to the cart, and complete the checkout
 - The Admin can manage the inventory



Use Case Diagram Tools

- draw.io
- Microsoft Visio

Case Study: A Career Guidance Portal

This portal is being developed to provide guidance to students, in answering papers in competitive exams. Guidance includes answering practice question papers and sending questions to counsellors, to which they get replies.

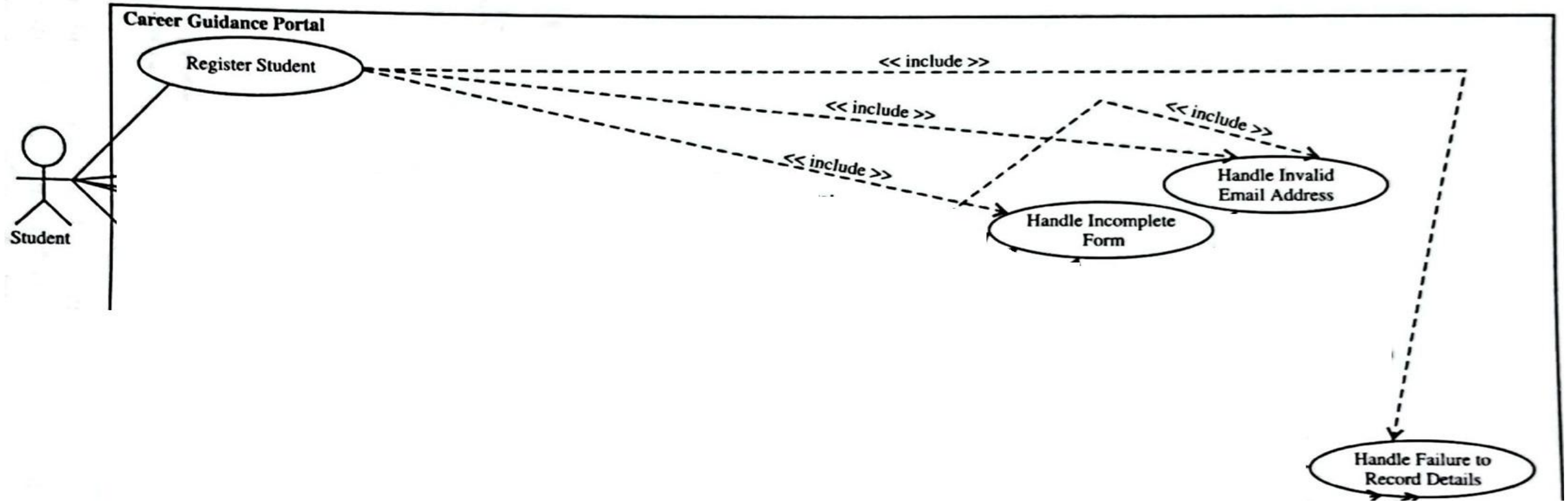
The student has to register with the portal, then he will be given a login id and a password. Registration is free and compulsory for people who wish to use the facilities provided by the portal. Whenever the student wants to answer a paper, he has to login, select

a paper of his choice and pay the fee online. The administrator will dispatch a key of the paper via email. The student has to follow the link to the paper, and use this key to open and answer it within the stipulated time. The system will process the answers and give a detailed summary of the student's performance, indicating the questions which were answered correctly/wrongly or not answered, and indicate the correct answers. If the student wants to post a query to a guidance counsellor, he can query the counsellor who can reply to the query through an email.

The portal generates revenue by displaying advertisements. A potential advertiser has to register with the portal, get a login-id, and a password. Using this id and password, the advertiser will log in, choose one of the possible schemes, and submit his advertisement to the administrator for approval. The administrator will either give his consent, or refuse to upload the advertisement. If the consent is given, the advertiser will have to pay through a credit card. On receiving the payment, the administrator will upload the advertisement on the scheduled date and time.

A potential counsellor has to register with the portal. When he fills the form and submits it, the administrator decides whether to give consent for his registration. If consent is given, the counsellor will be assigned a login-id and a password, else he will be notified through email that his request for registration to the portal has been denied. If his registration is successful, then his name will appear in the list of counsellors on the portal. Using his id and password, the counsellor will log in, choose the counselling requests that the student has made, and reply to that request.

The test maintenance authority uploads test papers for the students through the administrator. There are a fixed number of TMAs accepted by the company, who will upload test papers over the Internet from different geographical areas. The TMAs are not full-time employees; they are part-time workers who are paid on a per-paper basis. The TMA logs in, using his own id and password. He submits the details of the paper to the administrator, to which he gets a response that his request is being processed. The administrator checks the details of the paper, and either grants, or refuses permission for its upload. If permission is granted, an email is sent to the TMA notifying him of the decision to upload the paper at a specified date and time, and the TMA's account is credited with the appropriate amount. If permission is denied, he is sent a message informing him of the denial.



UC 2.1: Register Student

Primary actor: Student

Stakeholders and interests

Student: Wants a user-friendly environment and proper steps for the registration process. Wants to be assigned a login id and password after registration to the portal.

Administrator: Wants correct and valid information to be entered by the student. Wants automatic generation of student login id and password.

Pre-condition: The home page is displayed.

Post-condition: The student is registered and an appropriate login id and password is emailed to him.

Main success scenario (or basic flow)

1. Student clicks the 'New Student Registration' link.
2. System displays the 'New Student Registration' form.
3. Student enters his/her name, email-id, date of birth, residential address, education details, and submits the registration form.
4. System records student details.
5. System sends a 'Registration Successful' message at the email address provided by the student during registration.
6. System displays 'Registration Successful' message.
7. Student closes the web page and leaves the system.

Extensions (or alternative flows)

- 3a. Incomplete form submission: *Handle incomplete form.*
- 3b. Invalid Email address: *Handle invalid email address.*
- 4a. Record details failure: *Handle failure to record details.*

Special requirement

Student should login at least once in a month, else student's record will be deleted.

